

04 — Logi i diagnostyka aplikacji

kubectl get • describe • logs • exec • port-forward

```
kubectl logs deployment/qa-web -f
```



Cel modułu

umieć sprawdzić, czy aplikacja działa

odczytać logi Poda i Deploymentu

zobaczyć eventy i szczegóły obiektu

wejść do kontenera i zweryfikować środowisko

zdiagnozować podstawowe problemy z dostępem do aplikacji



Workflow QA



```
kubectl get pods  
kubectl describe pod <pod-name>  
kubectl logs <pod-name>  
kubectl port-forward svc/qa-web 8080:80
```

Najpierw status. Potem szczegóły. Na końcu logi i test dostępu.



Status obiektów

```
kubectl get pods  
kubectl get pods -o wide  
kubectl get deployment  
kubectl get svc
```

Najczęściej zaczynamy od pytania: co działa, co nie działa i gdzie działa?



describe: szczegóły i eventy

```
kubectl describe pod <pod-name>  
kubectl describe deployment qa-web  
kubectl describe svc qa-web
```

describe pokazuje konfigurację obiektu, status, powiązania oraz eventy.

```
kubectl get events --sort-by=.lastTimestamp
```



Logi: podstawy

```
kubectl logs <pod-name>  
kubectl logs deployment/qa-web  
kubectl logs <pod-name> -c <container-name>
```

Jeśli Pod ma jeden kontener, nazwa kontenera zwykle nie jest potrzebna.



Logi: opcje przydatne dla QA

```
kubectl logs deployment/qa-web -f  
kubectl logs <pod-name> --tail=50  
kubectl logs <pod-name> --since=10m  
kubectl logs <pod-name> --previous
```

--previous jest bardzo przydatne przy CrashLoopBackOff, bo pokazuje logi poprzedniego uruchomienia kontenera.



Aplikacja testowa do logów

Prosty Deployment, który co kilka sekund zapisuje komunikat do logów.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: qa-logger
spec:
  template:
    spec:
      containers:
      - name: logger
        image: busybox:1.36
        command: ["sh", "-c", "while true; do date; echo QA log event; sleep 5; done"]
```



Lab: obserwacja logów

```
kubectl apply -f logger-deployment.yaml  
kubectl get pods  
kubectl logs deployment/qa-logger  
kubectl logs deployment/qa-logger -f
```

To najprostszy sposób pokazania, że logi są strumieniem zdarzeń aplikacji.



exec: wejście do kontenera

```
kubectl exec -it <pod-name> -- sh  
ps  
env  
ls -la  
cat /etc/os-release
```

exec służy do szybkiej weryfikacji środowiska uruchomieniowego kontenera.



Diagnostyka Service

```
kubectl get svc  
kubectl describe svc qa-web  
kubectl get endpoints  
kubectl port-forward svc/qa-web 8080:80
```

Jeśli Service nie ma endpointów, zwykle selector nie pasuje do etykiet Podów.



Typowe objawy i pierwsze kroki

Objaw	Pierwsza komenda	Czego szukamy
Pod Pending	describe pod	brak zasobów / volume
CrashLoopBackOff	logs --previous	błąd startu aplikacji
ImagePullBackOff	describe pod	błędny obraz / registry
Brak dostępu	get endpoints	selector / port / targetPort



Lab: diagnoza krok po kroku

```
kubectl get pods  
kubectl describe pod <pod-name>  
kubectl logs <pod-name>  
kubectl logs <pod-name> --previous  
kubectl get svc  
kubectl get endpoints  
kubectl port-forward svc/qa-web 8080:80
```

Ten zestaw komend rozwiązuje większość pierwszych problemów aplikacyjnych.



Dobre praktyki dla QA

zawsze zaczynaj od `kubectl get pods`

`describe` pokazuje powód problemu częściej niż logi

przy restartach sprawdzaj `logs --previous`

dla Service sprawdzaj endpoints

nie drukuj sekretów w logach aplikacji



Sprzątanie środowiska

```
kubectl delete -f logger-deployment.yaml  
kubectl delete deployment qa-web  
kubectl delete svc qa-web  
kubectl get all
```

Po labie warto zostawić klaster w znanym stanie.

